

MiniPrologProto Stack

Manual & Tutorial

Prolog Resolution Engine + Prototype OOP + Column Database + HTN Planner

Edition: 2026-01-02

1. What this project is

This repository bundles four layers that work together:

- 1) MiniPrologProto: a small Prolog-style resolution engine in Java (unification + backtracking).
- 2) Prototype OOP library: Prolog-level prototype objects (clone/slots/methods/send).
- 3) Column database: a simple in-memory column store implemented using the prototype layer.
- 4) HTN planner: an HTN (Hierarchical Task Network) planner that can query the column DB through a strategy context.

You can use everything from a terminal REPL, or from the Python (PyQt6) GUI.

2. Setup and running

2.1 Requirements

- Java (recommended: 11+).
- Python 3 + PyQt6 (only for the GUI).
- macOS/Linux/Windows should work; scripts are bash-oriented (macOS/Linux).

2.2 Build and scripts

From the Java project folder:

```
cd prolog_real_final
./build.sh          # compile Java to ./out
./selftest.sh       # run engine self-tests
./demo.sh           # run sample demo (append, json, etc.)
./repl.sh           # start interactive REPL (full stack by default)
```

2.3 REPL stacks

The REPL can run in two modes:

- sample: minimal sample predicates (good for core engine checks)
- full: loads prototype OOP + column DB + planner + demo domain predicates

```
./repl.sh --sample
./repl.sh          # full stack
# or inside REPL:
:stack sample
:stack full
```

2.4 Python GUI

Run the GUI:

```
python3 prolog_ide.py
```

In the GUI, use Start REPL / Stop REPL to run the Java REPL and interact from the console pane.

3. REPL user guide

3.1 Basic interaction

Statements end with a period (.). The REPL accepts facts, rules, directives, and queries.

Multiple solutions: type ';' and press Enter for the next answer, or '.' to stop.

```
?- member(X, [a,b,c]).  
X = a ;  
X = b ;  
X = c .
```

3.2 REPL commands

Commands (no trailing period needed):

```
:help  
:stack sample|full  
:consult path/to/file.pl  
:predicates  
:reset  
:quit (or :q / :exit)
```

3.3 Consulting files

A .pl file can contain facts, rules, and queries. The REPL loads clauses into the current program via assertz/1.

Tip: keep one clause per line; end each clause with a period.

```
:consult examples/family.pl
```

4. Prolog language and built-ins

4.1 Terms

Supported term forms:

- Atoms: foo, bar, hello_world
- Numbers: 1, 42, -7
- Variables: X, Y, Result (underscore '_' is anonymous)
- Compounds: parent(alice,bob)
- Lists: [], [a,b,c], [H|T]

4.2 Facts, rules, recursion

```
parent(alice,bob).  
parent(bob,carol).
```

```
ancestor(X,Y) :- parent(X,Y).  
ancestor(X,Y) :- parent(X,Z), ancestor(Z,Y).
```

```
?- ancestor(alice, who).
```

4.3 Control operators

The parser supports Prolog-style goal operators:

- Conjunction: A, B
- Disjunction: A ; B
- If-then-else: (Cond -> Then ; Else)
- Negation-as-failure: \+ Goal
- Cut: !

```
?- (true -> X=1 ; X=2).  
?- \+ member(z, [a, b]).
```

4.4 Arithmetic and comparisons

Arithmetic is evaluated with is/2. Supported arithmetic operators: +, -, *, /, //, mod and unary -.

Comparisons: ==, =\=, <, <=, >, >= (numeric).

```
?- X is 2+3*4.  
?- 7 == 3+4.  
?- 10 > 3.
```

4.5 Built-ins provided by this engine

Core:

- true/0, fail/0, =/2
- call/1, once/1

Aggregation:

- findall/3, bagof/3, setof/3

Dynamic database:

- assertz/1, retract/1

JSON and Java POJOs:

- json_parse/2, json_stringify/2
- from_json/3, to_json/2

Heap hygiene (for POJO refs):

- release_ref/1, clear_heap/0, heap_size/1

4.6 JSON examples

```
?- json_parse("{\"x\":1,\"y\":[2,3]}", T).  
?- json_stringify([a,b,c], S).
```

POJO mapping uses reflection (example classes Person/Address in src).

```
?- from_json("{\"name\":\"Ada\",\"age\":36}", "com.mycompany.prolog.Person", Ref),  
   to_json(Ref, JsonOut),  
   release_ref(Ref).
```

5. Prototype OOP layer

The OOP layer is implemented as Prolog predicates in OopPrologLib.java.

Objects are terms obj(N). Prototypes are atoms like objectProto, dbProto, tableProto, etc.

5.1 Initialize OOP

```
?- oop_init.  
true.
```

5.2 Clone, slots, and proto chain

Create an object from a prototype with clone/2, set slots with put/3, read with get/3.

```
?- oop_init,  
    clone(objectProto, 0),  
    put(0, foo, 123),  
    get(0, foo, V).
```

5.3 Methods and send/2

Define methods on a prototype using defmethod/4. Methods receive an explicit Self parameter.

Call methods with send(Object, Message).

```
?- oop_init,  
    retract_all(proto(pointProto,_)), assertz(proto(pointProto, objectProto)),  
  
    defmethod(pointProto, init(X,Y), Self,  
              ( put(Self, x, X), put(Self, y, Y) )),  
  
    defmethod(pointProto, move(DX,DY), Self,  
              ( get(Self, x, X0), get(Self, y, Y0),  
                X1 is X0+DX, Y1 is Y0+DY,  
                put(Self, x, X1), put(Self, y, Y1) )),  
  
    clone(pointProto, P),  
    send(P, init(10,20)),  
    send(P, move(3,4)),  
    get(P, x, X), get(P, y, Y).
```

6. Column database

The column DB is defined in ColumnDbLib.java and uses the OOP layer.

6.1 Initialize

```
?- oop_init, columndb_init.  
true.
```

6.2 Core concepts and data structures

Schema is a list of col(Name, Type, Opts) terms.

Rows are represented as row(Pairs) where Pairs is a list of =(Key,Value) terms.

```
Schema = [col(id,num,opts), col(name,atom,opts)].  
Row     = row([=(id,1), =(name,alice)]).
```

6.3 Create a database and table

```
?- oop_init, columndb_init,  
    clone(dbProto, DB),  
    send(DB, create_table(users,  
                          [col(id,num,opts), col(name,atom,opts)],  
                          Users)).
```

6.4 Insert rows

```
?- send(Users, insert(row([=(id,1),=(name,alice)]))),  
    send(Users, insert(row([=(id,2),=(name,bob)]))),  
    send(Users, insert_many([  
        row([=(id,3),=(name,carol)]),
```

```

        row(=(id,4),=(name,dave)])
    ])).

```

6.5 Query: select/4

Call select as send(Table, select(Proj, Filter, Opts, Rows)).

Projection (Proj): all | [col1,col2,...]

Filter: all | where(Col, eq|ge, Literal) | and(F1,F2)

Options: opts(limit(all)) | opts(limit(N))

```

?- send(Users, select(all, all, opts(limit(all)), Rows)).
?- send(Users, select([name], where(id, ge, 3), opts(limit(all)), Rows)).
?- send(Users, select(all,
    and(where(id, ge, 2), where(name, eq, carol)),
    opts(limit(all)),
    Rows)).

```

6.6 Count and introspection

```

?- send(DB, list_tables(Names)).
?- send(DB, describe_table(users, Schema)).
?- send(Users, count(where(id, ge, 3), N)).

```

6.7 Indexes

You can build a hash index on a column for equality filters.

```

?- send(Users, create_index(name, hash)),
   send(Users, select(all, where(name, eq, carol), opts(limit(all)), Rows)).

```

7. HTN planner

The HTN planner lives in PlannerStrategiesLib.java. The demo domain is in HtnDemoLib.java.

7.1 Initialize planner strategies

The planner strategies require the OOP layer. If you use DB-backed needs, initialize the column DB too.

```

?- oop_init, columndb_init, planner_param_init.
true.

```

7.2 Contexts and profiles (small / cached / indexed)

Create a planning context with new_ctx(DB, Profile, Ctx). Profiles choose runner + search strategy:

- small: plain runner + DFS search
- cached: cached runner + bounded DFS search
- indexed: auto-index-eq runner + bounded DFS search

```

?- clone(dbProto, DB),
   new_ctx(DB, small, Ctx).

```

7.3 Planning interface

The main call is send(Ctx, plan(Tasks, Plan)). Tasks is a list of task terms. Plan is a list of primitive tasks.

```

?- send(Ctx, plan([fulfill(1)], Plan)).

```

7.4 What you must define for a domain

The planner expects these predicates in the current program:

- primitive(Task). % Task is primitive (an action)

- op(Ctx, Task). % Preconditions for a primitive task
 - method(Ctx, Task, Sub). % Decomposition for a compound task
- If your domain uses database reads through need_rows/3, also define:
- design_query(Need, q(Table,Proj,Filter,Opts)).

7.5 Minimal HTN example (no database)

```
primitive(boil_water).
primitive(steeep_tea).
primitive(pour).

method(_Ctx, make_tea, [boil_water, steep_tea, pour]).

op(_Ctx, boil_water).
op(_Ctx, steep_tea).
op(_Ctx, pour).

?- oop_init, planner_param_init,
   clone(dbProto, DB), new_ctx(DB, small, Ctx),
   send(Ctx, plan([make_tea], Plan)).
```

7.6 Database-backed domain patterns

need_rows/3 asks the context to design a query for a Need term, then runs the query through the context runner.

```
% Example mapping:
design_query(order_lines(0),
  q(order_lines, all, where(order_id, eq, 0), opts(limit(all)))).

% In a method:
method(Ctx, pick(0), Tasks) :-
  need_rows(Ctx, order_lines(0), Rows),
  lines_to_reserves(0, Rows, Tasks).
```

7.7 Run the included demo plan

The demo domain seeds a DB with orders, order_lines, inventory, and carriers, then plans fulfill(1).

```
?- demo_plan(indexed, Plan).
?- demo_plan(cached, Plan).
?- demo_plan(small, Plan).
```

8. Cookbook

8.1 Quick end-to-end: DB + plan

```
?- demo_plan(indexed, Plan).
```

8.2 Build your own DB query and reuse it in planning

- 1) Create tables and insert rows as in Section 6.
- 2) Add design_query/2 clauses that map needs to q(...) terms.
- 3) Write method/3 rules that call need_rows/3 and convert rows into tasks.
- 4) Run send(Ctx, plan(Tasks, Plan)).

9. Troubleshooting

- "expected '.'" or "unexpected end of input": you likely forgot the final period.
- "Unknown predicate": the goal functor/arity is not defined (or you used an operator the engine does not implement).
- OOP/DB methods fail: ensure you called oop_init (and columndb_init for DB).
- Rows not found: double-check filter structure where(Column, eq|ge, Literal).
- Planner loops or no plan: check primitive/1 and method/3 coverage; try cached/indexed profiles or reduce recursion.

10. Reference

10.1 Key predicates by subsystem

Core engine (always available):

- true/0, fail/0, =/2, ! (cut), \+/1, ,/2, ;/2, ->/2
- call/1, once/1
- is/2, =:=/2, =\=/2, </2, =</2, >/2, >=/2
- findall/3, bagof/3, setof/3
- assertz/1, retract/1
- json_parse/2, json_stringify/2, from_json/3, to_json/2
- release_ref/1, clear_heap/0, heap_size/1

Prototype OOP (call oop_init first):

- clone/2, get/3, put/3, defmethod/4, send/2
- proto/2, slot/3, method/4, retract_all/1

Column DB (call oop_init, columndb_init):

- DB methods (via send/2): create_table/3, table/2, list_tables/1, describe_table/2
- Table methods (via send/2): insert/1, insert_many/1, select/4, count/2, create_index/2

Planner (call oop_init, planner_param_init):

- new_ctx/3, need_rows/3, htn_dfs_plan/3, htn_bdfs_plan/4
- Domain hooks you provide: primitive/1, method/3, op/2, (optional) design_query/2
- Demo helper: demo_plan/2, demo_seed/1